

Code Injection with Token Sequences

Document #: P3294R1
Date: 2024-07-16
Project: Programming Language C++
Audience: SG7, EWG
Reply-to: Andrei Alexandrescu, NVIDIA
<andrei@nvidia.com>
Barry Revzin
<barry.revzin@gmail.com>
Daveed Vandevoorde
<daveed@edg.com>

Contents

- [1 Revision History](#)
- [2 Introduction](#)
- [3 A Comparison of Injection Models](#)
 - [3.1 The Spec API](#)
 - [3.2 The CodeReckons API](#)
 - [3.3 String Injection](#)
 - [3.4 Fragments](#)
- [4 Token Sequences](#)
 - [4.1 Token Sequence Expression](#)
 - [4.2 Interpolating into a Token Sequence](#)
 - [4.3 Phase of Translation](#)
 - [4.4 Injection](#)
 - [4.5 Token Sequence Type](#)
 - [4.6 Implementation Status](#)
 - [4.7 Examples](#)
 - [4.8 Type Erasure](#)
 - [4.9 Logging Vector: Cloning a Type](#)
 - [4.10 Logging Vector II: Cloning with Modifications](#)
- [5 Scoped Macros](#)
 - [5.1 Design Approach](#)
 - [5.2 Forwarding](#)
 - [5.3 Assertion](#)
 - [5.4 Macro Parsing](#)
 - [5.5 Hygienic Macros](#)
 - [5.6 String Interpolation](#)
 - [5.7 Abbreviated Lambdas](#)
 - [5.8 A control flow operator](#)

§ 1 Revision History

Since [\[P3294R0\]](#):

- Changed syntax for introducing token sequences to `^{ ... }`
- Refined the interpolator syntax to `\(e)`, `\id(e)`, and `\tokens(e)` (parens mandatory in all cases)
- Dropped the `declare [ : e : ] splicer`
- Implemented much of the proposal with links to examples (including a new [type erasure example](#))
- Changed syntax for macros, since the parameters can just be `std::meta::info`, and extended discussion of them.

§ 2 Introduction

This paper is proposing augmenting [\[P2996R4\]](#) to add code injection in the form of token sequences.

We consider the motivation for this feature to some degree pretty obvious, so we will not repeat it here, since there are plenty of other things to cover here. Instead we encourage readers to read some other papers on the topic (e.g. [\[P0707R4\]](#), [\[P0712R0\]](#), [\[P1717R0\]](#), [\[P2237R0\]](#)).

§ 3 A Comparison of Injection Models

There are a lot of things that make code injection in C++ difficult, and the most important problem to solve first is: what will the actual injection mechanism look like? Not its syntax specifically, but what is the shape of the API that we want to expose to users? We hope in this section to do a thorough job of comparing the various semantic models we're aware of to help explain why we came to the conclusion that we came to.

If you're not interested in this journey, you can simply skip to the [next section](#).

Here, we will look at a few interesting examples for injection and how different models can implement them. The examples aren't necessarily intended to be the most compelling examples that exist in the wild. Instead, they're hopefully representative enough to cover a wide class of problems. They are:

1. Implementing the storage for `std::tuple<Ts...>`
2. Implementing `std::enable_if` without resorting to class template specialization
3. Implementing properties (i.e. given a name like `"author"` and a type like `std::string`, emit a member `std::string m_author`, a getter `get_author()` which returns a `std::string` `const&` to

that member, and a setter `set_author()` which takes a new value of type `std::string` `const&` and assigns the member).

4. Implementing the postfix increment operator in terms of an existing prefix increment operator (a pure boilerplate annoyance).

§ 3.1 The Spec API

In P2996, the injection API is based on a function `define_class()` which takes a range of spec objects. In P2996, we only currently have `data_member_spec` - but this can conceivably be extended to have a `meow_spec` function for more aspects of the C++ API. Hence the name.

But `define_class()` is a really clunky API, because invoking it is an expression - but we want to do it in contexts that want a declaration. So a simple example of injecting a single member `int` named `x` is:

```
struct C;
static_assert(is_type(define_class(^C,
    {data_member_spec{.name="x", .type=^int}})));
```

We are already separately proposing `consteval` blocks [P3289R0] and we would like to inject each spec more directly, without having to complete C in one go. As in:

```
struct C {
    consteval {
        inject(data_member_spec{.name="x", .type=^int});
    }
};
```

Here, `std::meta::inject` is a new metafunction that takes a spec, which gets injected into the context of the `consteval` block which began our evaluation as a side-effect.

We already think of this as an improvement. But let's go through several use-cases to expand the API and see how well it holds up.

§ 3.1.1 `std::tuple`

The tuple use-case was already supported by P2996 directly with `define_class()` (even though we think it would be better as a member pack), but it's worth just showing what it looks like with a direct injection API instead:

```
template <class... Ts>
struct Tuple {
    consteval {
        std::array types{^Ts...};
        for (size_t i = 0; i != types.size(); ++i) {
            inject(data_member_spec{.name=std::format("_{}", i),
                .type=types[i]});
        }
    }
};
```

```

    }
}
};

```

§ 3.1.2 `std::enable_if`

Now, `std::enable_if` has already been obsolete technology since C++20. So implementing it, specifically, is not entirely a motivating example. However, the general idea of `std::enable_if` as *conditionally* having a member type is a problem that has no good solution in C++ today.

The spec API along with injection does allow for a clean solution here. We would just need to add an `alias_spec` construct to get the job done:

```

template <bool B, class T=void>
struct enable_if {
    consteval {
        if (B) {
            inject(alias_spec{.name="type", .type=^T});
        }
    }
};

```

So far so good.

§ 3.1.3 Properties

Now is when the spec API really goes off the rails. We've shown data members and extended it to member aliases. But how do we support member functions?

We want to be able to add a property with a given name and type that adds a member of that type and a getter and setter for it. For instance, we want this code:

```

struct Book {
    consteval {
        property("author", ^std::string);
        property("title", ^std::string);
    }
};

```

to emit a class with two members (`m_author` and `m_title`), two getters that each return a `std::string const&` (`get_author()` and `get_title()`) and two setters that each take a `std::string const&` (`set_author()` and `set_title()`). Fairly basic properties.

We start by injecting the member:

```

consteval auto property(string_view name, meta::info type)
-> void

```

```

{
    inject(data_member_spec{.name=std::format("m_{}", name),
                           .type=type});

    // ...
}

```

Now, we need to inject two functions. We'll need a new kind of spec for that case. For the function body, we can use a lambda. Let's start with the getter:

```

constexpr auto property(string_view name, meta::info type)
-> void
{
    inject(data_member_spec{.name=std::format("m_{}", name),
                           .type=type});

    inject(function_member_spec{
        .name=std::format("get_{}", name),
        .body=^[](auto const& self) -> auto const& {
            return self./* ????? */;
        }
    });

    // ...
}

```

Okay. Uh. What do we return? For the title property, this needs to be `return self.m_title;`, but how do we spell that? We just... can't. We have our member right there (the `data_member_spec` we're injecting), so you might think we could try to capture it:

```

constexpr auto property(string_view name, meta::info type)
-> void
{
    auto member = inject(data_member_spec{
        .name=std::format("m_{}", name),
        .type=type
    });

    inject(function_member_spec{
        .name=std::format("get_{}", name),
        .body=^[member](auto const& self) -> auto const& {
            return self.[:member:];
        }
    });

    // ...
}

```

But that doesn't work - in order to splice member, it needs to be a constant expression - and it's not in this context.

Now, the body of the lambda isn't going to be evaluated in this constant evaluation, so it's possible to maybe come up with some mechanism to pass a context through - such that from the body we *can* simply splice member. We basically need to come up with a way to defer this instantiation.

For now, let's try a spelling like this:

```
constexpr auto property(string_view name, meta::info type)
    -> void
{
    auto member = inject(data_member_spec{
        .name=std::format("m_{}", name),
        .type=type
    });

    inject(function_member_spec{
        .name=std::format("get_{}", name),
        .body=defer(member, ^[]<std::meta::info M>(auto const& self) ->
            auto const& {
                return self.[:M:];
            })
    });

    // ...
}
```

and we can do something similar with the setter:

```
constexpr auto property(string_view name, meta::info type)
    -> void
{
    auto member = inject(data_member_spec{
        .name=std::format("m_{}", name),
        .type=type
    });

    inject(function_member_spec{
        .name=std::format("get_{}", name),
        .body=defer(member, ^[]<std::meta::info M>(auto const& self) ->
            auto const& {
                return self.[:M:];
            })
    });

    inject(function_member_spec{
        .name=std::format("set_{}", name),
        .body=defer(member, ^[]<std::meta::info M>(auto& self, typename
            [:type_of(M):] const& x) -> void {
                self.[:M:] = x;
            })
    });
}
```

```
});
}
```

Now we run into the next problem: what actual signature is the compiler going to inject for `get_author()` and `set_author()`? First, we're introducing this extra non-type template parameter which we have to know to strip off somehow. Secondly, we're always taking the object parameter as a deduced parameter. How does the API know what we mean by that?

```
struct Book {
    // do we mean this
    auto get_author(this Book const& self) -> auto const& { return
        self.m_author; }
    auto set_author(this Book& self, string const& x) -> void {
        self.m_author = x; }

    // or this
    auto get_author(this auto const& self) -> auto const& { return
        self.m_author; }
    auto set_author(this auto& self, string const& x) -> void {
        self.m_author = x; }
};
```

That is: how does the compiler know whether we're injecting a member function or a member function template? Our lambda has to be generic either way. Moreover, even if we actually wanted to inject a function template, it's possible that we might want some parameter to be dependent but *not* the object parameter.

Well, we could provide another piece of information to `function_member_spec`: the signature directly:

```
template <class T> using getter_type = auto() const -> T const&;
template <class T> using setter_type = auto(T const&) -> void;

constexpr auto property(string_view name, meta::info type)
    -> void
{
    auto member = inject(data_member_spec{
        .name=std::format("m_{}", name),
        .type=type
    });

    inject(function_member_spec{
        .name=std::format("get_{}", name),
        .signature=substitute(^getter_type, {^type}),
        .body=defer(member, ^[]<std::meta::info M>(auto const& self) ->
            auto const& {
                return self.[:M:];
            })
    });

    inject(function_member_spec{
        .name=std::format("set_{}", name),
```

```

        .signature=substitute(^setter_type, {^type}),
        .body=defer(member, ^[]<std::meta::info M>(auto& self, typename
        [:type_of(M):] const& x) -> void {
            self.[:M:] = x;
        })
    });
}

```

Which then maybe feels like the correct spelling is actually more like this, so that we can actually properly infer all the information:

```

constexpr auto property(string_view name, meta::info type)
-> void
{
    auto member = inject(data_member_spec{
        .name=std::format("m_{}", name),
        .type=type
    });

    // note that this type is structural
    struct Context {
        std::meta::info type;
        std::meta::info member;
    };
    auto ctx = Context{
        // get the type of the current context that we're injecting into
        .type=type_of(std::meta::current()),
        .member=member
    };

    inject(function_member_spec{
        .name=std::format("get_{}", name),
        .body=defer(context, ^[]<Context C>(){
            return [] (typename [:C.type:] const& self) -> auto const& {
                return self.[:C.member:];
            };
        })
    });

    inject(function_member_spec{
        .name=std::format("set_{}", name),
        .body=defer(context, ^[]<Context C>(){
            return [] (typename [:C.type:]& self, typename
            [:type_of(C.member):] const& x) -> void {
                self.[:C.member:] = x;
            };
        })
    });
}

```

That is, we create a custom context type that we pass in as a non-type template parameter into a lambda, so that it *it* can return a new lambda with all the types and names properly substituted in when that can

actually be made to work.

This solution... might be workable. But it's already pretty complicated and the problem we're trying to solve really isn't. As a result, we believe that the spec API is somewhat of a dead end when it comes to extending injection support.

§ 3.1.4 Disposition

It's hard to view favorably a design for the long-term future of code injection with which we cannot even figure out how to inject functions. Even if we could, this design scales poorly with the language: we need a library API for many language constructs, and C++ is a language with a lot of kinds. That makes for a large barrier to entry for metaprogramming that we would like to avoid.

Nevertheless, the spec API does have one thing going for it: it is quite simple. At the very least, we think we should extend the spec model in P2996 in the following ways:

- extend `data_member_spec` to support all data members (static/constexpr/inline, attributes, access, and initializer).
- add `alias_spec` and `base_class_spec`

These are the simple cases, and we can get a lot done with the simple cases, even without a full solution.

§ 3.2 The CodeReckons API

The [CodeReckons](#) approach provides a very different injection mechanism than what is in P2996 or what has been described in any of the metaprogramming papers. We can run through these three examples and see what they look like. Here, we will use the actual syntax as implemented in that compiler.

§ 3.2.1 `std::tuple`

The initial CodeReckons article provides [an implementation](#) for adding the data members of a tuple like so:

```
template <class... Ts>
struct tuple {
    % [](class_builder& b){
        int k = 0;
        for (type T : std::meta::type_list{^Ts...}) {
            append_field(b, cat("m", k++), T);
        }
    }();
};
```

This isn't too different from what we showed in the earlier section with `data_member_spec`. Different spelling and API, but it's the same model (`append_field` is equivalent to injecting a `data_member_spec`).

§ 3.2.2 `std::enable_if`

Likewise, we have just a [difference of spelling](#):

```
template <bool B, typename T=void>
struct enable_if {
    % [] (class_builder& b) {
        if (B) {
            append_alias(b, identifier{"type"}, ^T);
        }
    }();
};
```

§ 3.2.3 Properties

Here is where the CodeRekons approach differs greatly from the potential spec API, and it's worth examining how they [got it working](#):

```
constexpr auto property(class_builder& b, type type, std::string name) -
    > void {
    auto member_name = identifier{"m_" + name}.c_str();
    append_field(b, member_name, type);

    // getter
    {
        method_prototype mp;
        object_type(mp, make_const(decl_of(b)));
        return_type(mp, make_lvalue_reference(make_const(type)));

        append_method(b, identifier{"get_" + name}.c_str(), mp,
            [member_name] (method_builder& b) {
                append_return(b,
                    make_field_expr(
                        make_deref_expr(make_this_expr(b)),
                        member_name));
            });
    }

    // setter
    {
        method_prototype mp;
        append_parameter(mp, "x",
            make_lvalue_reference(make_const(type)));
        object_type(mp, decl_of(b));
        return_type(mp, ^void);

        append_method(b, identifier{"set_" + name}.c_str(), mp,
            [member_name] (method_builder& b) {
                append_expr(b,
                    make_operator_expr(
                        operator_kind::assign,

                        make_field_expr(make_deref_expr(make_this_expr(b)),
                            member_name),
```

```

                                make_decl_ref_expr(parameters(decl_of(b))[1])
                                ));
                                });
                                }
                                }

struct Book {
    % property(^std::string, "author");
    % property(^std::string, "title");
};

```

In this model, we have to provide the signature of the two member functions (via `method_prototype`), and the bodies of the two member functions are provided as lambdas. But the lambda bodies here are not the C++ code that will be evaluated at runtime - it's still part of the AST building process. We have to define, at the AST level, what these member functions do.

In the spec API, we struggled how to write a function that takes a string `const&` and whose body is `self.{member name} = x;`. Here, because we don't need to access any of our reflections as constant expressions, we can make use of them directly.

But the result is... extremely verbose. This is a lot of code, that doesn't seem like it would scale very well. The setter alone (which is just trying to do something like `self.m_author = x;`) is already 14 lines of code and is fairly complicated. We think it's important that code injection still look like writing C++ code, not live at the AST level.

Nevertheless, this API does actually work. Whereas the spec API is still, at best, just a guess.

§ 3.2.4 Postfix Increment

For postfix increment, we want to inject the single function:

```

auto operator++(int) -> T {
    auto tmp = *this;
    ++*this;
    return tmp;
}

```

We rely on the type to provide the correct prefix increment. With the CodeReckons API, that looks [like this](#):

```

constexpr auto postfix_increment(class_builder& b) -> void {
    method_prototype mp;
    append_parameter(mp, "x", ^int);
    object_type(mp, decl_of(b));
    return_type(mp, decl_of(b));

    append_method(b, operator_kind::post_inc, mp,
        [](method_builder& b){
            // auto tmp = *this;

```

```

        auto tmp = append_var(b, "tmp", auto_ty,
                               make_deref_expr(make_this_expr(b)));
        // ++*this;
        append_expr(b,
                     make_operator_expr(
                         operator_kind::pre_inc,
                         make_deref_expr(make_this_expr(b))));
        // return tmp;
        append_return(b, make_decl_ref_expr(tmp));
    });

    struct C {
        int i;

        auto operator++() -> C& {
            ++i;
            return *this;
        }

        % postfix_increment();
    };

```

As with the property example above, having an AST-based API is extremely verbose. It might be useful to simply compare the statement we want to generate with the code that we require to write to generate that statement:

```

// auto tmp = *this;
    auto tmp = append_var(b, "tmp", auto_ty,
                           make_deref_expr(make_this_expr(b)));

// ++*this;
    append_expr(b, make_operator_expr(operator_kind::pre_inc,
                                       make_deref_expr(make_this_expr(b))));

// return tmp;
    append_return(b, make_decl_ref_expr(tmp));

```

§ 3.2.5 Disposition

We believe an important goal for code injection is that the code being injected looks like C++. This is the best way to ensure both a low barrier to entry for using the facility as well as easing language evolution in the future. We do not want to have to add a mirror API to the reflection library for every language facility we add.

The CodeReckons API has the significant and not-to-be-minimized property that it, unlike the Spec API, works. It is also arguably easy to *read* the code in question to figure out what is going on. In our experiments with simply giving people code snippets to people with no context and asking them what the snippet does, people were able to figure it out.

However, in our experience it is pretty difficult to *write* the code precisely because it needs to be written at a different layer than C++ code usually is written in and the abstraction penalty (in terms of code length) is so large. We will compare this AST-based API to a few other ideas in the following sections to give a sense of what we mean here.

§ 3.3 String Injection

If we go back all the way to the beginning - we're trying to inject code. Perhaps the simplest possible model for how to inject code would be: just inject strings.

The advantage of strings is clear: everyone already knows how to build up strings. This makes implementing the three use-cases presented thus far is pretty straightforward.

§ 3.3.1 `std::tuple`

We could just do tuple this way:

```
template <class... Ts>
struct Tuple {
    consteval {
        std::array types{^Ts...};
        for (size_t i = 0; i != types.size(); ++i) {
            inject(std::format(
                "[[no_unique_address]] {} _{};",
                qualified_name_of(types[i]),
                i));
        }
    }
};
```

Note that here we even added support for `[[no_unique_address]]`, which we haven't done in either of the previous models. Although we could come up with a way to add it to either of the two previous APIs, the fact that with string injection we don't even have to come up with a way to do this is a pretty significant upside. Everything just works.

Now, this would work - we'd have to be careful to use `qualified_name_of` here to avoid any question of name lookup. But it would be better to simply avoid these questions altogether by actually being able to splice in the type rather than referring to it by name.

We can do that by very slightly extending the API to take, as its first argument, an environment. And then we can reduce it again by having the API itself be a format API:

```
template <class... Ts>
struct Tuple {
    consteval {
        std::array types{^Ts...};
        for (size_t i = 0; i != types.size(); ++i) {
            inject(
```

```

        {"type", types[i]}},
        "[[no_unique_address]] [:type:] _{};",
        i);
    }
};

```

§ 3.3.2 `std::enable_if`

This one is even simpler, since we don't even need to bother with name lookup questions or splicing:

```

template <bool B, class T=void>
struct enable_if {
    consteval {
        if (B) {
            inject("using type = T;");
        }
    };
};

```

§ 3.3.3 Properties

Unlike with the spec API, implementing a property by way of code is straightforward. And unlike the CodeReckons API, we can write what looks like C++ code:

```

consteval auto property(info type, string_view name) -> void {
    inject(meta::format_with_environment(
        {"T", type}},
        R"(
        private:
            [:T:] m_{0};

        public:
            auto get_{0}() const -> [:T:] const& {{
                return m_{0};
            }}

            auto set_{0}(typename [:T:] const& x) -> void {{
                m_{0} = x;
            }}
        )",
        name));
}

struct Book {
    consteval {
        property(^string, "author");
        property(^string, "title");
    }
}

```

§ 3.3.4 Postfix Increment

Similarly, the postfix increment implementation just writes itself. In this case, we can even return `auto` so don't even need to bother with how to spell the return type:

```
constexpr auto postfix_increment() -> void {
    inject(R"(
        auto operator++(int) {
            auto tmp = *this;
            ++*this;
            return tmp;
        }
    )");
}

struct C {
    int i;

    auto operator++() -> C& {
        ++i;
        return *this;
    }

    constexpr { postfix_increment(); }
};
```

§ 3.3.5 Disposition

Can pretty much guarantee that strings have the lowest possible barrier to entry of any code injection API. Which is a benefit that is not to be taken lightly! It is not surprising that D and Jai both have string-based injection mechanisms.

But string injection is hardly perfect, and several of the issues with it might be clear already:

1. String injection does let you write what looks like C++ code, but it wouldn't let you use any macros - as those don't affect the contents of string literals and we can't run another preprocessing step later.
2. Our main string formatting mechanism, `format`, uses `{}` for replacement fields, which means actual braces - which show up in C++ a lot - have to be escaped. It also likely isn't the most compile-time efficient API, so driving reflection off of it might be suboptimal.
3. You don't get syntax highlighting for injected code strings. They're just strings. Perhaps we could introduce a new kind of string literal that syntax highlighters could recognize, but that seems like pre-emptively admitting defeat.
4. Errors happen at the point of *injection*, not at the point where you're writing the code. And the injection could happen very far away from the code.
5. There is no natural way to interpolate reflection values, and that is quite desirable (e.g. we attempted to use `qualified_name_of()` to inject a type name, but that's not robust - and

qualified_name_of() is hard to get right anyway).

But string injection offers an extremely significant advantage that's not to be underestimated: everyone can deal with strings and strings already just support everything, for all future evolution, without the need for a large API.

Can we do better?

§ 3.4 Fragments

[P1717R0] introduced the concept of fragments. It introduced many different kinds of fragments, under syntax that changed a bit in [P2050R0] and [P2237R0]. We'll use what the linked implementation uses, but feel free to change it as you read.

§ 3.4.1 std::tuple

The initial fragments paper itself led off with an implementation of std::tuple storage and the concept of a `constexpr` block (now also [P3289R0]). That [looks like this](#) (the linked implementation looks a little different, due to an implementation bug):

```
template<class... Ts>
struct Tuple {
    constexpr {
        std::array types{^Ts...};
        for (size_t i = 0; i != types.size(); ++i) {
            -> fragment struct {
                [[no_unique_address]] typename(%{types[i]})
                unqualid("_", %{i});
            };
        }
    };
};
```

Now, the big advantage of fragments is that it's just C++ code in the middle there (maybe it feels a bit messy in this particular example, but it will be more clear in other examples). The leading `->` is the injection operator.

One big problem that fragments need to solve is how to get context information into them. For instance, how do we get the type `types[i]` and how do we produce the names `_0`, `_1`, ..., for all of these members? We need a way to capture context, and it needs to be interpolated differently.

In the above example, the design uses the operator `unqualid` (to create an unqualified id) concatenating the string literal `"_"` with the interpolated value `%{i}` (a later revision used `|# #|` instead). We need distinct operators to differentiate between the case where we want to use a string as an identifier and as an actual string.

```
std::string name = "hello";
-> fragment struct {
```



```
// std::string name = "name";
std::string unqualid(#{name}) = #{name};
};
```

§ 3.4.2 std::enable_if

It is very hard to compete [with this](#):

```
template <bool B, class T=void>
struct enable_if {
    consteval {
        if (B) {
            -> fragment struct { using type = T; };
        }
    };
};
```

Sure, you might want to simplify this just having a class scope `if` directly and then putting the contents of the fragment in there. But this is very nice.

§ 3.4.3 Properties

The [implementation here](#) isn't too different from the [string implementation](#) (this was back when the reflection operator was `reflexpr`, before it changed to `^`):

```
consteval auto property(meta::info type, char const* name) -> void {
    -> fragment struct {
        typename(#{type}) unqualid("m_", #{name});

        auto unqualid("get_", #{name})() -> typename(#{type}) const& {
            return unqualid("m_", #{name});
        }

        auto unqualid("set_", #{name})(typename(#{type}) const& x) ->
        void {
            unqualid("m_", #{name}) = x;
        }
    };
}

struct Book {
    consteval {
        property(reflexpr(std::string), "author");
        property(reflexpr(std::string), "title");
    }
};
```

It's a bit busy because nearly everything in properties involves interpolating outside context, so seemingly everything here is interpolated.

Now, there's one very important property that fragments (as designed in these papers) adhere to: every fragment must be parsable in its context. A fragment does not leak its declarations out of its context; only out of the context where it is injected. Not only that, we get full name lookup and everything.

On the one hand, this seems like a big advantage: the fragment is checked at the point of its declaration, not at the point of its use. With the string model above, that was not the case - you can write whatever garbage string you want and it's still a perfectly valid string, it only becomes invalid C++ code when it's injected.

On the other, it has some consequences for how we can code using fragments. In the above implementation, we inject the whole property in one go. But let's say we wanted to split it up for whatever reason. We can't. This is invalid:

```
constexpr auto property(meta::info type, char const* name) -> void
{
    -> fragment struct {
        typename(%{type}) unqualid("m_", %{name});
    };

    -> fragment struct {
        auto unqualid("get_", %{name})() -> typename(%{type}) const& {
            return unqualid("m_", %{name}); // error
        }

        auto unqualid("set_", %{name})(typename(%{type}) const& x) ->
        void {
            unqualid("m_", %{name}) = x; // error
        }
    };
}
```

In this second fragment, name lookup for `m_author` fails in both function bodies. We can't do that. We have to teach the fragment how to find the name, which requires writing this (note the added `requires` statement):

```
constexpr auto property(meta::info type, char const* name) -> void
{
    -> fragment struct {
        typename(%{type}) unqualid("m_", %{name});
    };

    -> fragment struct {
        requires typename(%{type}) unqualid("m_", %{name});

        auto unqualid("get_", %{name})() -> typename(%{type}) const& {
            return unqualid("m_", %{name}); // error
        }

        auto unqualid("set_", %{name})(typename(%{type}) const& x) ->
        void {

```

```

        unqualid("m_", %{name}) = x; // error
    }
};
}

```

§ 3.4.4 Postfix increment

Postfix increment ends up being [much simpler to implement](#) with fragments than properties - due to not having to deal with any interpolated names. But it does surface the issue of name lookup in fragments.

```

consteval auto postfix_increment() {
    -> fragment struct T {
        requires T& operator++();

        auto operator++(int) -> T {
            auto tmp = *this;
            ++*this;
            return tmp;
        }
    };
}

struct C {
    int i;

    auto operator++() -> C& {
        ++i;
        return *this;
    }

    consteval { postfix_increment(); }
};

```

Now, the rule in the fragments implementation is that the fragments themselves are checked. This includes name lookup. So any name used in the body of the fragment has to be found and pre-declared, which is what we're doing in the `requires` clause there. The implementation right now appears to have a bug with respect to operators (if you change the body to calling `inc(*this)`, it does get flagged), which is why it's commented out in the link.

§ 3.4.5 Disposition

The fragment model seems substantially easier to program in than the CodeReckons model. We're actually writing C++ code. Consider the difference here between the CodeReckons solution and the Fragments solution to postfix increment:

CodeReckons	Fragments
<pre> consteval auto postfix_increment(class_builder& b) -> void { </pre>	<pre> consteval auto postfix_increment() { -> fragment struct T { </pre>

CodeReckons	Fragments
<pre> method_prototype mp; append_parameter(mp, "x", ^int); object_type(mp, decl_of(b)); return_type(mp, decl_of(b)); append_method(b, operator_kind::post_inc, mp, [](method_builder& b){ auto tmp = append_var(b, "tmp", auto_ty, make_deref_expr(make_this_expr(b))); append_expr(b, make_operator_expr(operator_kind::pre_inc, make_deref_expr(make_this_expr(b)))); append_return(b, make_decl_ref_expr(tmp)); }); } struct C { int i; auto operator++() -> C& { ++i; return *this; } % postfix_increment(); }; </pre>	<pre> requires T& operator++ (); auto operator++(int) - > T { auto tmp = *this; ++*this; return tmp; } }; struct C { int i; auto operator++() -> C& { ++i; return *this; } consteval { postfix_increment(); } }; </pre>

We lined up the fragment implementation to roughly correspond to the CodeReckons API on the left. With the code written out like this, it's easy to understand the CodeReckons API. But it takes no time at all to understand (or write) the fragments code on the right - it's just C++ already.

We also think it's a better idea than the string injection model, since we want something with structure that isn't just missing some parts of the language (the preprocessor) and plays nicely with tools (like syntax highlighters).

But we think the fragment model still isn't quite right. By nobly trying to diagnose errors at the point of fragment declaration, it adds a complexity to the fragment model in a way that we don't think carries its weight. The fragment papers ([[P1717R0](#)] and [[P2237R0](#)]) each go into some detail of different approaches of how to do name checking at the point of fragment declaration. They are all complicated.

We basically want something between strings and fragments.

§ 4 Token Sequences

Generation of code from low-level syntactic elements such as strings or token sequences may be considered quite unsophisticated. Indeed, previous proposals for code synthesis in C++ have studiously avoided using strings or tokens as input, instead resorting to AST-based APIs, expansion statements, or code fragments, as shown above. As noted by Andrew Sutton in [P2237R0]:

synthesizing new code from strings is straightforward, especially when the language/library has robust tools for compile-time string manipulation [...] the strings or tokens are syntactically and semantically unanalyzed until they are injected

whereas the central premise—and purported advantage—of a code fragment is it

should be fully syntactically and semantically validated prior to its injection

Due to the lack of consensus for a code synthesis mechanism, some C++ reflection proposals shifted focus to the query side of reflection and left room for scant code synthesis capabilities.

After extensive study and experimentation (as seen above), we concluded that a form of token-based synthesis is crucially important for practical code generation, and that insisting upon early syntactic and semantic validation of generated code is a net liability. The very nature of code synthesis involves assembling meaningful constructs out of pieces that have little or no meaning in isolation. Using concatenation and deferring syntax/semantics analysis to offer said concatenation is by far the simplest, most direct approach to code synthesis.

Generally, we think that imposing early checking on generated code is likely to complicate and restrict the ways in which users can use the facility — particularly when it comes to composing larger constructs from smaller ones — and also be difficult for implementers, thus hurting everyone involved.

We therefore choose the notion of *token sequence* as the core building block for generating code. Unparsed token sequences allow for flexible composition, while deferring semantic analysis (lookup, etc.) to the point of injection avoids complexities in trying to re-create the context of the point of injection at the point of composition.

§ 4.1 Token Sequence Expression

We propose the introduction of a new kind of expression with the following syntax (the specific introducer can be decided later):

```
^{ balanced-brace-tokens }
```

where *balanced-brace-tokens* is an arbitrary sequence of C++ tokens with the sole requirement that the `{` and `}` pairs are balanced. Parentheses and square brackets may be unbalanced. The opening and closing `{/}` are not part of the token sequence. The type of a token sequence expression is `std::meta::info`.

The choice of syntax is motivated by two notions:

1. If we could reflect on the body of a function template, the only thing that could be yielded back is a token sequence - since the template hasn't been instantiated yet. And the syntax for reflecting on that body would look like `^{ body }`
2. This maintains the property that the only built-in operator that produces a `std::meta::info` value is the prefix `^`.

For example:

```
constexpr auto t1 = ^{ a + b };           // three tokens
static_assert(std::is_same_v<decltype(t1), const std::meta::info>);
constexpr auto t2 = ^{ a += ( );         // code does not have to be
    meaningful
constexpr auto t3 = ^{ abc { def };      // Error, unpaired brace
```

[Editor's note: We are aware of the conflict with Objective-C/C++ blocks that makes this syntax untenable. For now, the paper is written still using `^` and a subsequent version will have to find something else, probably still choosing the same prefix operator as reflection.]

§ 4.2 Interpolating into a Token Sequence

There's still the issue that we need to access outside context from within a token sequence. For that we introduce dedicated interpolation syntax using three kinds of interpolators:

- `\(expression)`
- `\id(string, string-or-intopt...)`
- `\tokens(expression)`

The implementation model for this is that we collect the tokens within a `^{ ... }` literal, but every time we run into an interpolator, we parse the expressions within. When the token sequence is evaluated (always a compile-time operation since it produces a `std::meta::info` value), the expressions are evaluated and the corresponding interpolators are replaced as follows:

- `\id(e)` for `e` being string-like is replaced with that string as a new *identifier*. `\id(e...)` can concatenate multiple string-like or integral values into a single *identifier* (the first argument must be string-like).
- `\(e)` is replaced by a pseudo-literal token holding the value of `e`. The parentheses are mandatory.
- `\tokens(e)` is replaced by the — possibly empty — tokens `e` (`e` must be a reflection of an evaluated token sequence).

The value and `id` interpolators need to be distinct because a given string could be intended to be injected as a *string*, like `"var"`, or as an *identifier*, like `var`. There's no way to determine which one is intended, so they have to be spelled differently.

We initially considered `+` for token concatenation, but we need token sequence interpolation anyway. Consider wanting to build up the token sequence `T{a, b, c}` where `a`, `b`, `c` is the contents of another token sequence. With interpolation, that is straightforward:

```
^{ T{\tokens(args)} } }
```

but with concatenation, we run into a problem:

```
^{ T{ } } + args + ^{ } }
```

This doesn't produce the intended effect because it is a token sequence containing the tokens `T { } + args + ^ { }` instead of an expression containing two additions involving two token sequences as desired.

Given that we need `\tokens` anyway, additionally adding concatenation with `+` and `+=` doesn't seem as necessary, especially since keeping the proposal minimal has a lot of value.

Using `\` as an interpolator has at least some prior art. Swift uses `\(e)` in their [string interpolation syntax](#).

§ 4.2.1 Alternate Interpolation Syntax

Currently, we are proposing three interpolators: `\`, `\id`, and `\tokens`. That might seem like a lot, especially `\tokens` is a lot of characters, but we feel that this is the complete necessary set. A simple alternative is to spell `\tokens(e)` instead as `\{e}` (i.e. braces instead of parentheses). This is a lot shorter, but it's still three interpolators (and the visual distinction might be too subtle).

A bigger alternative would be to overload interpolation on types. In Rust, for instance, interpolation into a procedural macro always is spelled `#var` - and opting in to interpolation is implementing the trait [ToTokens](#). The way to interpolate an identifier is to interpolate an object of type `syn::Ident`. Going that route (and making tokens sequences [their own type](#)) might mean that the approach becomes:

```
auto seq = ^{  
- auto \id("_", x) = \tokens(e);  
+ auto \(\std::meta::token::id("_", 1)) = \{e};  
};
```

Or, with a handy using-directive or using-declaration:

```
auto seq = ^{  
- auto \id("_", x) = \tokens(e);  
+ auto \(\id("_", 1)) = \{e};  
};
```

This loses some orthogonality, namely what if we want to inject a *value* of type `token_sequence`. But for that we can always resort to `\(reflect_value(tokens))`, which is probably a rare use-case.

§ 4.3 Phase of Translation

Token sequences are a construct that is processed in translation phase 7 (5.2 [\[lex.phases\]](#)). This has some natural consequences detailed below.

The result of interpolating with `\tokens` is a token sequence consisting of all the tokens of both sequences:

```
constexpr auto t1 = ^{ c = };
constexpr auto t2 = ^{ a + b; };
constexpr auto t3 = ^{ \tokens(t1) \tokens(t2) };
static_assert(t3 == ^{ c = a + b; });
```

It is unclear if we want to support `==` for token sequences, but it is easier to express the intent if we use it. So this paper will use `==` at least for exposition purposes.

The concatenation is not textual - two tokens concatenated together preserve their identity, they are not pasted together into a single token. For example:

```
constexpr auto t1 = ^{ abc };
constexpr auto t2 = ^{ def };
constexpr auto t3 = ^{ \tokens(t1) \tokens(t2) };
static_assert(t3 != ^{ abcdef });
static_assert(t3 == ^{ abc def });
```

Whitespace and comments are treated just like in regular code - they are not significant beyond their role as token separator. For example:

```
constexpr auto t1 = ^{ hello = /* world */ "world" };
constexpr auto t2 = ^{ /* again */ hello="world" };
static_assert(t1 == t2);
```

Tokens are handled after the initial phases of preprocessing: macros and string concatenation can apply, but occur before the implementation assembles a token sequence. You therefore have to be careful with macros because they won't work the way you might want to:

```
constexpr auto operator+(info t1, info t2) -> info {
    return ^{ \tokens(t1) \tokens(t2) };
}

static_assert(^{ "abc" "def" } == ^{ "abcdef" });
```



```

// this concatenation produces the token sequence "abc" "def", not
// "abcdef"
// when this token sequence will be injected, that will be ill-formed
static_assert(^{ "abc" } + ^{ "def" } != ^{ "abcdef" });

#define PLUS_ONE(x) ((x) + 1)
static_assert(^{ PLUS_ONE(x) } == ^{ ((x) + 1) });

// amusingly this version also still works but not for the reason you
// think
// on the left-hand-side the macro PLUS_ONE is still invoked...
// but as PLUS_ONE(x} +^{)
// which produces ((x} +^{) + 1)
// which leads to ^{ ((x } + ^{) + 1) }
// which is ^{ ((x) + 1)}
static_assert(^{ PLUS_ONE(x } + ^{ ) } == ^{ PLUS_ONE(x) });

// But this one finally fails, because the macro isn't actually invoked
constexpr auto tok2 = []{
    auto t = ^{ PLUS_ONE(x } ;
    constexpr_print_str("Logging...\n");
    t += ^{ ) }
    return t;
}();
static_assert(tok2 != ^{ PLUS_ONE(x) });

```

A token sequence has no meaning by itself, until injected. But because (hopefully) users will write valid C++ code, the resulting injection actually does look like C++ code.

§ 4.4 Injection

Once we have a token sequence, we need to do something with it. We need to inject it somewhere to get parsed and become part of the program.

We propose two injection functions.

`std::meta::queue_injection(e)`, where `e` is a token sequence, will queue up a token sequence to be injected at the end of the current constant evaluation - typically the end of the `consteval` block that the call is made from.

`std::meta::namespace_inject(ns, e)`, where `ns` is a reflection of a namespace and `e` is a token sequence, will immediately inject the contents of `e` into the namespace designated by `ns`.

We can inject into a namespace since namespaces are open - we cannot inject into any other context other than the one we're currently in.

As a [simple example](#):

```

#include <experimental/meta>

constexpr auto f(std::meta::info r, int val, std::string_view name) {

```

```

    return ^{ constexpr [:\(r):] \id(name) = \(val); };
}

constexpr auto r = f(^int, 42, "x");

namespace N {}

consteval {
    // this static assertion will be injected at the end of the block
    queue_injection(^{ static_assert(N::x == 42); });

    // this declaration will be injected right into ::N right now
    namespace_inject(^N, r);
}

int main() {
    return N::x != 42;
}

```

With that out of the way, we can now go through our examples from earlier.

§ 4.5 Token Sequence Type

In this paper (and the current implementation), the type of a token sequence is also `std::meta::info`. This follows the general [P2996R4] design that all types that are opaque handles into the compiler have type `std::meta::info`. And that is appealing for its simplicity.

However, unlike reflections of source constructs, token sequence manipulation is a completely disjoint set of operations. The only kinds of reflection that can produce token sequences can only ever produce token sequences (e.g. getting the `noexcept` specifier of a function template).

Some APIs only make sense to do on a token sequence - for instance while we described `+` as not being essential, we could certainly still provide it - but from an API perspective it'd be nicer if it took two objects of type `token_sequence` rather than two of type `info` (and asserted that they were `token_sequences`). Either way, misuse would be a compile error, but it might be better to only provide the operator when we know it's viable.

A dedicated `token_sequence` type would also make macros (as introduced [below](#)) stand out more from other reflection functions, since there will be a lot of functions that take a `meta::info` and return a `meta::info` and such functions are quite different from macros.

§ 4.6 Implementation Status

A significant amount of this proposal is already implemented in EDG and is available for experimentation on Compiler Explorer. The examples we will demonstrate provide links.

The implementation provides a `__report_tokens(e)` function that can be used to dump the contents of a token sequence during constant evaluation to aid in debugging.

Two things to note with the implementation:

- While we intend `\id("hello", 1)` to work, currently the string-like pieces must actually have type `std::string_view` - `\id("hello"sv, 1)` does work and will produce the identifier `hello1`.
- Injecting into a class template is currently very limited. Attempts to inject member function definitions will lead to linker errors and attempts to inject nested class definitions will fail. Currently, this will require using `namespace_inject` to inject the entire class template specialization in one go. You can see this approach in action with the [type erasure example](#).

§ 4.7 Examples

Now, the `std::tuple` and `std::enable_if` cases look nearly-identical to their corresponding implementations with [fragments](#). In both cases, we are injecting complete code fragments that require no other name lookup, so there is not really any difference between a token sequence and a proper fragment.

[Implementing `Tuple<Ts...>`](#) requires using both the value interpolator and the identifier interpolator (in this case we're naming the members `_0`, `_1`, etc.):

```
template <class... Ts>
struct Tuple {
    consteval {
        std::meta::info types[] = {^Ts...};
        for (size_t i = 0; i != sizeof...(Ts); ++i) {
            queue_injection(^{ [[no_unique_address]] [:(types[i]):]
                \id("_", i); });
        }
    }
};
```

whereas [implementing `enable\_if<B, T>`](#) doesn't require any interpolation at all:

```
template <bool B, class T=void>
struct enable_if {
    consteval {
        if (B) {
            queue_injection(^{ using type = T; });
        }
    }
};
```

The property example likewise could be identical to the fragment implementation, but we do not run into any name lookup issues, so we can write it any way we want - either as injecting one token sequence or even injecting three. Both work fine without needing any additional declarations.

But we may want to restrict injection to one declaration at a time for error reporting purposes (this is currently enforced by the EDG implementation).

That implementation [looks like this](#):

```
constexpr auto property(std::meta::info type, std::string_view name)
    -> void
{
    auto member = ^{ \id("m_"sv, name) };

    queue_injection(^{ [:\(type):] \tokens(member); });

    queue_injection(^{
        auto \id("get_"sv, name)() -> [:\(type):] const& {
            return \tokens(member);
        }
    });

    queue_injection(^{
        auto \id("set_"sv, name)(typename [:\(type):] const& x)
            -> void {
            \tokens(member) = x;
        }
    });
}

struct Book {
    constexpr {
        property(^std::string, "title");
        property(^std::string, "author");
    }
};
```

With the postfix increment example, we see some more interesting difference. We are not proposing any special-case syntax for getting at the type that we are injecting into, so it would have to be pulled out from the context (we'll name it T in both places for consistency):

Fragment	Token Sequence
<pre>constexpr auto postfix_increment() -> void { -> fragment struct T { requires T& operator++(); auto operator++(int) -> T { auto tmp = *this; ++*this; return tmp; } }; }</pre>	<pre>constexpr auto postfix_increment() -> void { auto T = std::meta::nearest_class_or_namespace(); queue_injection(^{ auto operator++(int) -> [:\(T):] { auto tmp = *this; ++*this; return tmp; } }); }</pre>

The syntax here is, unsurprisingly, largely the same. We're mostly writing C++ code. The difference is that we no longer need to pre-declare the functions we're using and the feature set is smaller. While declaring T as part of the fragment is certainly convenient, we're shooting for a smaller feature.

§ 4.8 Type Erasure

Given a type, whose declaration only contains member functions that aren't templates, it is possible to mechanically produce a type-erased version of that interface.

For instance:

Interface	Type-Erased
<pre>struct Interface { void draw(std::ostream&) const; };</pre>	<pre>template <> class Dyn<Interface> { struct VTable { // 1. convert each function in Interface to // a // function pointer with an extra void* void (*draw)(void*, std::ostream&); }; template <class T> static constexpr VTable vtable_for = VTable { // 2. convert each function in Interface to // a // forwarding, static-casting lambda .draw = +[](void* data, std::ostream& p0) - > void { // NB: the const here because // Interface::draw() is const return static_cast<T const*>(data)- >draw(p0); } }; VTable const* vtable; void* data; public: template <class T> // 3. convert each function in Interface to // its // appropriate requires clause // NB: the remove_cvref_t<T> const here // because // Interface::draw() is const requires requires (std::remove_cvref_t<T> const t, std::ostream& p0) { { t.draw(p0) } -> std::convertible_to<void>; } Dyn(T&& t)</pre>

Interface	Type-Erased
	<pre> : vtable(&vtable_for<std::remove_cvref_t<T>>) , data(&t) { } Dyn(Dyn&) = default; Dyn(Dyn const&) = default; ~Dyn() = default; // 4. convert each function in Interface to a // function // that forwards through the vtable auto draw(std::ostream& p0) const -> void { return vtable->draw(data, p0); } }; </pre>

That implementation is currently non-owning, but it isn't that much of a difference to make it owning, move-only, have a small buffer optimized storage, etc.

There is a lot of code on the right (especially compared to the left), but the transformation is *purely* mechanical. It is so mechanical, in fact, that it lends itself very nicely to precisely the kind of code injection being proposed in this paper.

You can find the implementation [here](#). Note that the current implementation uses `namespace_inject` to produce the entire template specialization of `Dyn`. We hope to not have to require that approach, but at the moment EDG cannot inject nested type definitions in a class template. It's a healthy amount of code, but it's actually fairly straightforward.

§ 4.9 Logging Vector: Cloning a Type

The goal here is we want to implement a type `LoggingVector<T>` which behaves like `std::vector<T>` in all respects except that it prints the function being called.

We start with this:

```

template <typename T>
class LoggingVector {
    std::vector<T> impl;

public:
    LoggingVector(std::vector<T> v) : impl(std::move(v)) { }

    consteval {
        for (std::meta::info fun : /* public, non-special member
            functions */) {
            queue_injection(^{
                \tokens(make_decl_of(fun)) {
                    // ...
                }
            });
        }
    };
}

```

```

    }
}
};

```

We want to clone every member function, which requires copying the declaration. We don't want to actually have to spell out the declaration in the token sequence that we inject - that would be a tremendous amount of work given the complexity of C++ declarations. But the nice thing about token sequence injection is that we really only have to do that *one* time and stuff it into a function. `make_decl_of()` can just be a function that takes a reflection of a function and returns a token sequence for its declaration. We'll probably want to put this in the standard library.

Now, we have two problems to solve in the body (as well as a few more problems we'll get to later).

First, we need to print the name of the function we're calling. This is easy, since we have the function and can just ask for its name.

Second, we need to actually forward the parameters of the function into our member impl. This is, seemingly, very hard:

```

constexpr {
    for (std::meta::info fun : /* public, non-special member functions
        */) {
        queue_injection(^{
            \tokens(make_decl_of(fun)) {
                std::println("Calling {}", \name_of(fun));
                return impl.[:\fun:](/* ??? */);
            }
        });
    }
}

```

This is where the ability of token sequences to be concatenated from purely sequences of tokens really gives us a lot of value. How do we forward the parameters along? We don't even have the parameter names here - the declaration that we're cloning might not even *have* parameter names.

So there are two approaches that we can use here:

§ 4.9.1 Reflections of Parameters

We need the ability to just ask for the parameters themselves (which [\[P3096R1\]](#) should provide). And then the goal here is to inject the tokens for the call:

```

return impl.[:fun:]([:p0:], [:p1:], ..., [:pn:])

```

But the tricky part is that we can't ask for the parameters of the function we're cloning (i.e. `fun` in the loop above - which is a reflection of a non-static member function of `std::vector<T>`), we have to ask

for the parameters of the function that we're *currently* defining. Which we haven't defined yet so we can't reflect on it.

But we could split this in pieces and ask inject to give us back a reflection of what it injected, since inject must operate on full token boundaries.

So that might be:

```
template <typename T>
class LoggingVector {
    std::vector<T> impl;

public:
    LoggingVector(std::vector<T> v) : impl(std::move(v)) { }

    consteval {
        for (std::meta::info fun : /* public, non-special member
            functions */) {
            // note that this one doesn't even require a token sequence
            auto log_fun = queue_injection(decl_of(fun));

            // convenience type for building a comma-delimited sequence
            auto argument_list = list_builder();

            for (auto param : parameters_of(log_fun)) { // <== NB, not
                fun
                argument_list += ^{
                    static_cast<[:\ (type_of(param)):]&&>[: \ (param) :])
                };
            }

            queue_injection(^{
                \tokens(make_decl_of(fun)) {
                    std::println("Calling {}", \ (name_of(fun)));
                    return impl.[:\ (fun):]( [: \tokens(argument_list):]
                );
            });
        }
    }
};
```

The argument_list is simply building up the token sequence `[: p0 :]`, `[: p1 :]`, ..., `[: pN :]` for each parameter (except forwarded). There is no name lookup going on, no checking of fragment correctness. Just building up the right tokens.

Once we have those tokens, we can concatenate this token sequence using the same interpolator that we've used for other problems and then splice them in. In the same way that splicing a reflection of a type produces a type, splicing a reflection of a token sequence produces those tokens.

Note that we didn't actually have to implement it using a separate `argument_list` local variable - we could've concatenated the entire token sequence piecewise. But this structure allows factoring out parameter-forwarding into its own function:

```
constexpr auto forward_parameters(std::meta::info fun) ->
    std::meta::info {
    auto argument_list = list_builder();
    for (auto param : parameters_of(fun)) {
        argument_list += ^{
            static_cast<[:\ (type_of(param)):]&&> ([:\param:])
        };
    }
    return argument_list;
}
```

And then:

```
constexpr {
    for (std::meta::info fun : /* public, non-special member functions
        */) {
        auto log_fun = queue_injection(decl_of(fun));

        queue_injection(^{
            \tokens(make_decl_of(fun)) :] {
                std::println("Calling {}", \ (name_of(fun)));
                return impl.[:\ (fun):] ( [:
            \tokens(forward_parameters(log_fun)) :] );
            }
        });
    }
}
```

The problem is - this direction isn't really viable. Injection queues up requests for later. It may not be feasible for us to get back a reflection of `log_fun` in the way that we are using this example, so we probably cannot actually get back and access the reflections of the parameters as described in this example.

§ 4.9.2 Introducing Parameter Names

We said we have the problem that the functions we're cloning might not have parameter names. So what? We're creating a new function, we can pick our names!

Since our approach to cloning function declarations is just writing our function that creates the tokens:

```
\tokens(make_decl_of(fun)) { /* ... */ }
```

We can simply pass another argument to `make_decl_of` that gives us a prefix for each parameter name. So maybe `make_decl_of(fun, "p")` would give us parameter names of `p0`, `p1`, and so forth. That gives us a similar looking solution, but now we never need the reflection of the new function - just the old one:

```
template <typename T>
class LoggingVector {
    std::vector<T> impl;

public:
    LoggingVector(std::vector<T> v) : impl(std::move(v)) { }

    consteval {
        for (std::meta::info fun : /* public, non-special member
            functions */) {
            auto argument_list = list_builder();
            for (size_t i = 0; i != parameters_of(fun).size(); ++i) {
                argument_list += ^{
                    // we could get the nth parameter's type (we can't
                    // splice
                    // the other function's parameters but we CAN query
                    // them)
                    // or we could just write decltype(p0)
                    static_cast<decltype(\id("p", i))&&>(\id("p", i))
                };
            }

            queue_injection(^{
                \tokens(make_decl_of(fun, "p")) {
                    std::println("Calling {}", \name_of(fun));
                    return impl.[:\(fun):]( [::\(argument_list):] );
                }
            });
        }
    };
};
```

This approach is arguably simpler than reflecting on parameter names and requires no extra implementation effort to get there.

§ 4.10 Logging Vector II: Cloning with Modifications

However, we've still got some work to do. The above implementation already gets us a great deal of functionality, and should create code that looks something like this:

```
template <typename T>
class LoggingVector {
    std::vector<T> impl;

public:
    LoggingVector(std::vector<T> v) : impl(std::move(v)) { }
```

```

auto clear() -> void {
    std::println("Calling {}", "clear");
    return impl.clear();
}

auto push_back(T const& value) -> void {
    std::println("Calling {}", "push_back");
    return impl.push_back(static_cast<T const&>(value));
}

auto push_back(T&& value) -> void {
    std::println("Calling {}", "push_back");
    return impl.push_back(static_cast<T&&>(value));
}

// ...
};

```

For a lot of `std::vector`'s member functions, we're done. But some need some more work. One of the functions we're emitting is member swap:

```

template <typename T>
class LoggingVector {
    std::vector<T> impl;

public:
    // ...

    auto swap(std::vector<T>& other) noexcept(/* ... */) -> void {
        std::println("Calling {}", "swap");
        return impl.swap(other); // <== omitting the cast here for
                                // readability
    }

    // ...
};

```

But this... isn't right. Or rather, it could potentially be right in some design, but it's not what we want to do. We don't want `LoggingVector<int>` to be swappable with `std::vector<int>`... we want it to be swappable with itself. What we actually want to do is emit this:

```

auto swap(LoggingVector<T>& other) noexcept(/* ... */) -> void {
    std::println("Calling {}", "swap");
    return impl.swap(other.impl);
}

```

Two changes here: the parameter needs to change from `std::vector<T>&` to `LoggingVector<T>&`, and then in the call-forwarding we need to forward not `other` (which is now the wrong type) but rather

`other.impl`. How can we do that? We don't quite have a good answer yet. But this is much farther than we've come with any other design.

§ 5 Scoped Macros

C macros have a (well-deserved) bad reputation in the C++ community. This is because they have some intractable problems:

- C macros don't follow any scoping rules, and can change any code, anywhere. This is why they do not leak into or out of C++ modules.
- The C preprocessor is a language unto itself, that doesn't understand C++ syntax, with limited functionality that is very tedious to program in. Even what we would consider to be very basic language constructs like `if` or `for` are expert-level features in the preprocessor, and even then are highly limited.

We think that C++ does need a code manipulation mechanism, and that token sequences can provide a much better solution than C macros.

§ 5.1 Design Approach

One way to think about a macro is that it is a function that takes *code* and produces *code*, without necessarily evaluating or even parsing the code (indeed the code that is input to the macro need not even be valid C++ at all).

With token sequences, we suddenly gain a way to represent macros in C++ proper: a macro is a function that takes a token sequence and returns a token sequence, whereby it can be automatically injected (with some syntax marker at the call site).

This is already implicitly the way that macros operate in LISPs like Scheme and Racket, and is explicitly how they work in Rust and Swift. In Rust, [procedural macros](#) have the form:

```
#[proc_macro]
pub fn macro(input: TokenStream) -> TokenStream {
    ...
}
```

Whereas in Swift, [macros](#) have the form ([proposal](#)):

```
public struct FourCharacterCode: ExpressionMacro {
    public static func expansion(
        of node: some FreestandingMacroExpansionSyntax,
        in context: some MacroExpansionContext
    ) throws -> ExprSyntax {
        ...
    }
}
```

```
}  
}
```

Either way, unevaluated raw code in, unevaluated raw code out.

Now that we have the ability to represent code in code (using token sequences) and can inject said code that is produced by regular C++ functions, we can do in the same in C++ as well.

§ 5.2 Forwarding

Consider the problem of forwarding. Forwarding an argument in C++, in the vast majority of uses, looks like `std::forward<T>(t)`, where `T` is actually the type `decltype(t)`. This is annoying to write, the operation is simply forwarding an argument but we have to duplicate that argument nonetheless. And it requires the instantiation of a template (although compilers are moving towards making that a builtin).

Barry at some point proposed a specific language feature for this use-case ([P0644R1]). Later, there was a proposal for a hygienic macro system [P1221R1] in which forwarding would be implemented like this:

```
using fwd(using auto x) {  
    return static_cast<decltype(x)&&>(x);  
}  
  
auto old_f = [](auto&& x) { return std::forward<decltype(x)>(x); };  
auto new_f = [](auto&& x) { return fwd(x); };
```

With token sequences, using the design described earlier that we accept code in and return code out, we can achieve similar syntax:

```
constexpr auto fwd2(meta::info x) -> meta::info {  
    return ^{  
        static_cast<decltype([:\tokens(x):])&&>([:\tokens(x):]);  
    };  
}  
  
auto new_f2 = [](auto&& x) { return fwd2!(x); };
```

The logic here is that `fwd2!(x)` is syntactic sugar for `immediately_inject(fwd2(^{ x })))` (which requires a new mechanism for injecting into an expression). We're taking a page out of Rust's book and suggesting that invoking a "macro" with an exclamation point does the injection. Seems nice to both have convenient syntax for token manipulation and a syntactic marker for it on the call-site.

The first revision of this paper used the placeholder syntax `@tokens x` to declare the parameter of `fwd2`, but it turns out that this is just a token sequence - so it can just have type `std::meta::info`. The call-site syntax of `fwd2!` should be all you need to request tokenization.

Of course, fwd2 is a regular C++ function. You have to invoke it through the usual C++ scoping rules, so it does not suffer that problem from C macros. And then the body is a regular C++ function too, so writing complex token manipulation is just a matter of writing complex C++ code - which is a lot easier than writing complex C preprocessor code.

Note that the invocation of a macro like `macro!(std::pair<int, int>{1, 2})` would just work fine - the argument passed to macro would be `^{ std::pair<int, int>{1, 2} }`. But that leads us to the question of parsing...

§ 5.3 Assertion

Consider a different example (borrowed from [here](#)):

```
constexpr auto assert_eq(meta::info a, meta::info b) -> meta::info {
    return ^{
        do {
            auto sa = \(stringify(a));
            auto va = \tokens(a);

            auto sb = \(stringify(b));
            auto vb = \tokens(b);

            if (not (va == vb)) {
                std::println(
                    stderr,
                    "{} ({{}) == {} ({{}) failed at {}",
                    sa, va,
                    sb, vb,
                    \source_location_of(a));
                std::abort();
            }
        } while (false);
    };
}
```

With the expectation that:

Written Code	Injected Code
<code>assert_eq!(42, factorial(3));</code>	<pre>do { auto sa = "42"; auto va = 42; auto sb = "factorial(3)"; auto vb = factorial(3); if (not (va == vb)) { std::println(stderr, "{} ({{}) == {} ({{}) failed at {}", sa, va,</pre>

Written Code	Injected Code
	<pre> sb, vb, /* some source location */; std::abort(); } } while(false); </pre>

You can write this as a regular C macro today, but we bet it's a little nicer to read using this language facility.

However, this macro brings up two problems that we have to talk about: parsing and hygiene.

§ 5.4 Macro Parsing

The signature of the `assert_eq!` macro we have above was:

```
consteval auto assert_eq(meta::info a, meta::info b) -> meta::info;
```

Earlier we described the design as taking *a single* token sequence and producing a token sequence output. We'd of course want to express `assert_eq` as a function taking two token sequences, but how does the compiler know when to end one token sequence and start the next? That requires parsing. If the user writes `assert_eq!(std::pair<int, int>{1, 2}, x)`, the compiler needs to figure out which comma in there is actually an argument delimiter (or how to fail if there is only one argument).

There are a couple ways that we could approach this.

We could always require that a macro takes a single token-sequence argument and provide a parser library to help pull out the pieces. For instance, [in Rust](#), you would write something like this:

```

// Parse a possibly empty sequence of expressions terminated by commas
// with
// an optional trailing punctuation.
let parser = Punctuated::<Expr, Token![,]>::parse_terminated;
let _args = parser.parse(tokens)?;

```

And then for `assert_eq!`, verify that there are two such expressions and then do the rest of the work.

Alternatively, we could push this more into the signature of the macro - choosing how to tokenize the input based on the parameter type list:

```

// this parses f!(1+2, f(3, 4))
// into f({1+2}, {f(3, 4)})
consteval auto f(meta::token::expr lhs, meta::token::expr rhs) ->
    meta::info;

// this parses g!(1+2, f(3, 4))

```

```
// into g({ 1+2, f(3, 4) })
consteval auto g(meta::info xs) -> meta::info;

// this parses h!(1+2, f(3, 4))
// into h!({ ^{1+2}, ^{f(3, 4)}})
// so that xs.size() == 2
consteval auto h(meta::token::expr_list xs) -> meta::info
```

The last example here with `h` is roughly the same idea as the parser example - except changing who does what work, where.

§ 5.5 Hygienic Macros

Regardless of how we parse the two expressions that are input into our macro, this still suffers from at least one C macro problem: naming. If instead of `assert_eq!(42, factorial(3))` we wrote `assert_eq!(42, sa * 2)`, then this would not compile - because name lookup in the `do-while` loop would end up finding the local variable `sa` declared by the macro.

There are broadly two approaches to solve this problem:

Macros are hygienic by default: names introduced in macros are (at least by default) distinct from names that are injected into those macros. This is the case in Racket and Scheme, as well as declarative Macros in Rust. For instance, in Rust, this code:

```
macro_rules! using_a {
    ($e:expr) => {
        {
            let a = 42;
            $e
        }
    }
}

let four = using_a!(a / 10);
```

emits

```
let four = {
    let a = 42;
    a / 10
}
```

Note that the two `as` are spelled the same, but one is orange. That coloring is how hygienic macros work - names get an extra kind of scope depending on where they are used. So here the `a` in the `using_a` macro is in a different *span* than the `a` in the `a / 10` tokens that were passed into the macro, so they are considered different names.

Sometimes an unhygienic macro is useful though, to deliberately create an *anaphoric macro*. The canonical example is wanting to write an anaphoric if which takes an expression and, if it's truthy, passes that expression as the name it to the then callable:

```
(aif #t (displayln it) (void))
```

Scheme/Racket have syntax-rules to be able to provide such an unhygienic parameter.

A more familiar example of an anaphoric macro in C++ would be the ability to declare a unary lambda whose parameter is named it in a very abbreviated form, as in:

```
auto positive = std::ranges::count_if(r, λ!(it > 0));
```

which we can declare as:

```
constexpr auto λ(meta::info body) -> meta::info {  
    return ^{  
        [&](auto&& it)  
            noexcept(noexcept(\tokens(body)))  
            -> decltype(\tokens(body))  
        {  
            return \tokens(body);  
        }  
    }  
}
```

Such a macro would not work in a hygienic system, because the `it` in the expression `it > 0` would not find the parameter declared `it` as they live in different spans.

Alternatively, macros are *not* hygienic by default. This is the case for Rust procedural macros, Swift's macros, and to a very extreme degree, C. In order to make unhygienic macros usable, you need *some* mechanism of coming up with unique names if the language won't do it for you. The LISP approach to this is a function named `gensym` which generates a unique symbol name. This takes more effort on the macro writer (who has to remember to use `gensym`) when they want hygienic variables - which is likely the overwhelmingly common case, unlike the anaphoric case in a hygienic system where the macro writer needs to opt out of hygiene.

With hygienic macros, the assertion example is already correct. With unhygienic macros, we'd need to do something like this:

```
constexpr auto assert_eq(meta::info a, meta::info b) -> meta::info {  
    auto [sa, va, sb, vb] = std::meta::make_unique_names<4>();  
  
    return ^{  
        do {
```

```

    auto \id(sa) = \(\stringify(a));
    auto \id(va) = \tokens(a);

    auto \id(sb) = \(\stringify(b));
    auto \id(vb) = \tokens(b);

    if (not (\id(va) == \id(vb))) {
        std::println(
            stderr,
            "{} ({{}}) == {{}} ({{}}) failed at {}",
            \id(sa), \id(va),
            \id(sb), \id(vb),
            \(\source_location_of(a)));
        std::abort();
    }
} while (false);
};
}

```

That is, all the uses of local variables like `va` instead turn into `\id(va)`. It's not a huge amount of work, but it does get you into the same level of ugliness that we're used to seeing in standard library implementations with all uses of `__name` instead of `name` to avoid collisions. Although this particular example might oversell the issue, since `sa` and `sb` don't really need to be local variables - we could have just directly formatted `\(\stringify(a))` and `\(\stringify(b))`, respectively.

Obviously, an unhygienic system is much easier to implement and specify - since hygiene would add complexity (and likely some overhead) to how name lookup works.

§ 5.6 String Interpolation

Many programming languages support string interpolation. The ability to write something like `format!("x={x}")` instead of `format("x={{}}", x)`. It's a pretty significant feature when it comes to the ergonomics of formatting.

We can write it as a library:

```

// the actual parsing isn't interesting here.
// the goal is to take a string like "x={this->x:02} y={this->y:02}"
// and return {.format_str="x={:02} y={:02}", .args={"this->x", "this->y"}}}
struct FormatParts {
    string_view format_str;
    vector<string_view> args;
};

consteval auto parse_format_string(string_view) -> FormatParts;

consteval auto format(string_view str) -> meta::info {
    auto parts = parse_format_string(str);

    auto tok = ^{

```

```

        // NB: there's no close paren yet
        // we're allowed to build up a partial fragment like this
        ::std::format\(parts.format_str)
    };

    for (string_view arg : parts.args) {
        tok = ^{ \tokens(tok), \tokens(tokenize(arg)) };
    }

    // now finally here's our close paren
    return ^{ \tokens(tok) } };
}

```

In the previous example, we demonstrated the need for a way to convert a token sequence to a string. In this example, we need a way to convert a string to a token sequence. This doesn't involve parsing or any semantic analysis. It's *just* lexing.

Of course, this approach has limitations. We cannot fully faithfully parse the format string because at this layer we don't have types - we can't stop and look up what type `this->x` was, instantiate the appropriate `std::formatter<X>` and use it tell us where the end of its formatter is. We can just count balanced `{}`s and hope for the best.

Similarly, something like `format!("{SOME_MACRO(x)}")` can't work since we're not going to rerun the preprocessor during tokenization. But I doubt anybody would even expect that to work.

But realistically, this would handily cover the 90%, if not the 99% case. Not to mention could easily adopt other nice features of string interpolation that show up in other languages (like Python's `f"{x =}"` which formats as `"x = 42"`) as library features. And, importantly, this isn't a language feature tied to `std::format`. It could easily be made into a library to be used by any logging framework.

Note here that unlike previous examples, the `format` macro just took a `string_view`. This is in contrast to the earlier examples where the macro had to take a token sequence (possibly with some [parsing](#) involved). Depending on how we approach parsing, the design could simply be that any implicit tokenization only occurs if the macro's parameters actually expect token sequences. Or it could be that the `format!` macro needs to take a token sequence too and parse a string literal out of it.

§ 5.7 Abbreviated Lambdas

In the hygiene section, we had an example of an abbreviated, unary lambda using a parameter named `it`. That is something that could already be done in a C macro today. However, one thing that cannot easily be done in a C macro is to generalize this to writing a lambda macro that can take a specified number of parameters. As in:

```

consteval auto λ(int n, meta::info body) -> meta::info {
    // our parameters are _1, _2, ..., _n
    auto params = list_builder();
    for (int i = 0; i < n; ++i) {
        params += ^{ auto&& \id("_", i+1) };
    }
}

```

```

    }

    // and then the rest is just repeating the body
    return ^{
        [&](\tokens(params))
            noexcept(noexcept(\tokens(body)))
            -> decltype(\tokens(body))
        {
            return \tokens(body);
        }
    };
}

```

As with the string interpolation example, here we're now taking one parameter of type `int` (that doesn't need to be tokenized) and another parameter that are the actual tokens. The usage here might be something like `λ!(2, _1 > _2)` - which is a lambda version of `std::greater{}`.

Of course it'd be nice to do even better. That is: we can infer the arity of the lambda based on the parameters that are used. This paper does not yet have an API for iterating over a token sequence - but this particular problem would not involve parsing. Simply iterate over the tokens and find the largest `n` for which there exists an identifier of the form `_n` and use that as the arity. That would allow `λ!(_1 > _2)` by itself to be a binary lambda (or a lambda that takes at least two parameters). Can't do that with a C macro!

§ 5.8 A control flow operator

Two papers currently in flight propose extensions to C++'s set of expressions: [\[P2806R2\]](#) proposes `do` expressions as a way to have multiple statements in a single expression, and [\[P2561R2\]](#) proposes a control flow operator for better ergonomics with types like `std::expected<T, E>`.

Now, the proposed control flow operator nearly lowers into a `do` expression - with one exception that is covered [in the paper](#): lifetime. It would be nice if `f().try?`, for a function returning `expected<T, E>`, evaluated to `T&&` rather than `T` - to save an unnecessary move. But doing so requires actually storing that result... somewhere. What if macro injection allowed us to create such a somewhere?

```

// an extremely lightweight Optional, only for use in deferring storage
template <class T>
struct Storage {
    union { T value; }; // assume P3074 trivial union
    bool initialized = false;

    constexpr ~Storage() {
        if (initialized) {
            value.~T();
        }
    }
}

template <class F>
constexpr auto construct(F f) -> T& {
    assert(not initialized);
}

```

```

        auto p = new (&value) T(f());
        initialized = true;
        return *p;
    }
};

consteval auto try_(meta::info body) -> meta::info {
    // 1. we need the type of the body
    meta::info T = type_of(body);

    // 2. we create a local variable in the nearest enclosing scope
    //     that is of type Storage<T>
    meta::info storage = create_local_variable(substitute(^Storage,
        {T}));

    return ^{
        do -> decltype(auto) {
            // 3. we construct the "body" of the macro into that storage
            auto& r = [: \ (storage) :].construct(
                [&]() -> decltype(auto) { return (\tokens(body)); }
            );

            // 4. and then do the usual dance with returning the error
            if (not r) { return std::move(r).error(); }
            do_return *std::move(r);
        }
    }
}

```

There is plenty of novelty here. First, we need to get the type of the body. `body` are just some tokens - this might be called like `try_!(f(1, 2))` or `try_!(var)`, and we want `decltype(f(1, 2))` and `decltype(var)`, respectively, as evaluated from the context where the macro was invoked. Actually what we really want is `decltype((f(1, 2)))` and `decltype((var))`, respectively. For now, we'll use the existing `type_of` as a placeholder to achieve that type.

Second, `create_local_variable` returns a reflection to an unnamed (and thus not otherwise accessible) local variable that is created as close as possible to the injection site, of the provided type (which must be default constructible). This of course opens the door for lots of havoc, but in this case gives us a convenient place to just grab some storage that we need for later.

Once we have those two pieces, the rest is actually straightforward. The body of the `do` expression constructs our expected `<T, E>` into the local storage we just carved out, and then uses it directly. We do all of this dance instead of just `auto&& r = \tokens(body);` simply to be able to return a reference from the `do` expression.

Importantly though, macros coupled with this kind of storage injection allows [\[P2561R2\]](#) to be shipped as a library.

§ 5.9 Operator Support

One advantage of the trailing `!` syntax used here is that it provides a clear signal to the compiler and the reader that something new is going on. Using such a syntax means we cannot support operators though `x &&! y` already has valid meaning today, and it is not macro-invoking `operator&&`.

If we want to support operators (and we are not sure if we do), then one approach would be to introduce a new syntax for a macro declaration (which we may want to do anyway). Such a macro could work like this:

```
struct C {
    bool b;

    macro operator&&(this std::meta::info self, std::meta::info rhs) {
        return ^{ [:\(self):].b && \tokens(rhs); }
    }
};

auto x = C{false} && some_call();
```

Here, the macro would evaluate `C{false}` and pass a reflection to that expression as the first parameter, then the second parameter is just tokenized. Thus the call effectively evaluates as `C{false}.b && some_call()`, which does short-circuit as desired.

It's unclear if macro operators are worth pursuing. Dedicated macro syntax declarations might be beneficial though.

§ 5.10 Alternate Syntax

We have two forms of injection in this paper:

- metafunctions `std::meta::queue_injection` and `std::meta::namespace_inject` that take an `info`, used through [token sequences](#).
- a trailing `!` used for [scoped macros](#).

But these really are similar - both are requests to take a token sequence and inject it in the current context. The bigger token sequence injection doesn't really have any particular reason to require terse syntax. Prior papers did use some punctuation marks (e.g. `->`, `<<`), but a named function seems better. But the macros *really* do want to have terse invocation syntax. Having to write `immediately_inject(forward(x))` somewhat defeats the purpose and nobody would write it.

Using one of the arrows for the macro use-case is weird, so one option might be prefix `@`. As in `@forward(x)`, `@assert_eq(a, b)`, and `@format("x={this->x}")`. This is what Swift does, except using prefix `#` (which isn't really a viable option for us as `#x` already has meaning in the existing C preprocessor and we wouldn't want to completely prevent using new macros inside of old macros).

Or we could stick with two syntaxes - the longer one for the bigger reflection cases where terseness is arguably bad, and the short one for the macro use case where terseness is essential.

Likewise, macros could be declared as regular functions that take a token sequence and return a token sequence (or [other parameters](#)). Or perhaps we introduce a new context-sensitive keyword instead:

```
// regular function
constexpr auto fwd(meta::info x) -> meta::info { return ^{ /* ... */ };
}

// dedicated declaration
macro fwd(meta::info x) { return ^{ /* ... */ }; }
```

§ 6 Proposal

We propose a code injection mechanism using token sequences.

The fragment model initially introduced in [P1717R0] is great for allowing writing code-to-be-injected to actually look like regular C++ code, which has the benefit of being both familiar and being already recognizable to tools like syntax highlighters. But the early checking adds complexity to the model and the implementation which makes it harder to use and limits its usefulness. Hence, we propose raw token sequences that are unparsed until the point of injection.

This proposal consists of several pieces:

- a mechanism to introduce a token sequence (in this paper `^{ balanced-brace-tokens }`)
- three interpolators to add outside context to a token sequence, one for identifiers (`\id(e...)`), one for values (`\(e)` - parens mandatory), and one for token sequences (`\tokens(e)`)
- new metafunctions to inject a token sequence (`std::meta::queue_injection()` and `std::meta::namespace_inject()`)
- new metaprogramming facilities for dealing with token sequences:
 - converting a string to a token sequence and a token sequence to a string
 - splitting a token sequence into a range of tokens and querying/mutating those tokens
- macros would benefit syntactically from:
 - a mechanism to accept a tokens sequence as a function parameter
 - a mechanism to inject a token sequence directly as returned by a function (trailing `!`)

Note that the macro proposal, and even the facilities for splitting/iterating/querying/mutating tokens, can be split off as well. We feel that even the core proposal of injecting token sequences in declaration contexts only can provide a tremendous amount of value.

§ 7 References

[P0644R1] Barry Revzin. 2017-10-08. Forward without forward.
<https://wg21.link/p0644r1>

- [P0707R4] Herb Sutter. 2019-06-17. Metaclasses: Generative C++.
<https://wg21.link/p0707r4>
- [P0712R0] Andrew Sutton, Herb Sutter. 2017-06-18. Implementing language support for compile-time programming.
<https://wg21.link/p0712r0>
- [P1221R1] Jason Rice. 2018-10-03. Parametric Expressions.
<https://wg21.link/p1221r1>
- [P1717R0] Andrew Sutton, Wyatt Childers. 2019-06-17. Compile-time Metaprogramming in C++.
<https://wg21.link/p1717r0>
- [P2050R0] Andrew Sutton, Wyatt Childers. 2020-01-13. Tweaks to the design of source code fragments.
<https://wg21.link/p2050r0>
- [P2237R0] Andrew Sutton. 2020-10-15. Metaprogramming.
<https://wg21.link/p2237r0>
- [P2561R2] Barry Revzin. 2023-05-18. A control flow operator.
<https://wg21.link/p2561r2>
- [P2806R2] Barry Revzin, Bruno Cardoso Lopez, Zach Laine, Michael Park. 2023-11-16. do expressions.
<https://wg21.link/p2806r2>
- [P2996R4] Wyatt Childers, Dan Katz, Barry Revzin, Andrew Sutton, Faisal Vali, and Daveed Vandevoorde. 2024-06-26. Reflection for C++26.
<https://wg21.link/p2996r4>
- [P3096R1] Adam Lach, Walter Genovese. 2024-05-15. Function Parameter Reflection in Reflection for C++26.
<https://wg21.link/p3096r1>
- [P3289R0] Daveed Vandevoorde, Wyatt Childers, Barry Revzin. 2024-05-21. Consteval blocks.
<https://wg21.link/p3289r0>
- [P3294R0] Barry Revzin, Andrei Alexandrescu, Daveed Vandevoorde. 2024-05-22. Code Injection with Token Sequences.
<https://wg21.link/p3294r0>